

A Laboratory Report
on
Grouping Anagrams Using Strings and Sorting

Submitted to the Data Structures Laboratory
by

PASUMARTHY S S S M SURYATEJ KAASYAP (25261A04H6)
YADLA LOHITH (25261A04K1)

Course Instructor
Dr. Y. Praveen Kumar Reddy



Department of Electronics and Communication Engineering
Mahatma Gandhi Institute of Technology (Autonomous)
Chaitanya Bharathi P.O., Gandipet, Hyderabad-500075.

2026

Contents

1	Abstract	1
2	Introduction & Motivation	2
2.1	Introduction	2
2.2	Motivation	2
3	Design & Algorithms	3
3.1	Design Approach	3
3.2	Sorting Logic	3
3.3	Time Complexity	4
4	Implementation Details	5
4.1	Programming Language Used	5
4.2	Data Structures Used	5
4.3	Working Procedure	5
4.4	Key Functional Components	5
4.5	Assumptions	6
4.6	Limitations	6
5	Test Plan & Results	7
5.1	Testing Methodology	7
5.2	Test Case 1	7
5.3	Test Case 2	7
5.4	Test Case 3	8
5.5	Result Analysis	8
6	Analysis & Discussion	9
6.1	Performance Analysis	9
6.2	Discussion	9
6.3	Applications	9

7	Conclusion & Future Work	10
7.1	Conclusion	10
7.2	Future Work	10
8	Appendix & Code Snippets	11
8.1	Character Sorting Function	11
8.2	Grouping Logic	11
8.3	Sample Input and Output	12
9	References	13

1 Abstract

Anagrams are words formed by rearranging the letters of another word while preserving all original characters exactly once. Identifying and grouping anagrams is a common problem in string processing and algorithm design.

This project focuses on designing and implementing an efficient program to group a collection of words into corresponding anagram sets. The implementation is carried out using the C programming language with concepts such as arrays, strings, sorting techniques, and comparison operations.

The core idea behind the solution is to convert every word into a canonical form by sorting its characters alphabetically. Words sharing the same sorted representation are grouped together as anagrams.

The project demonstrates efficient data organization and string manipulation techniques while providing practical exposure to algorithmic problem solving. The developed solution achieves improved efficiency and produces organized grouped outputs for different input conditions.

2 Introduction & Motivation

2.1 Introduction

String processing plays an important role in computer science applications such as text analysis, search optimization, spell checking, and natural language processing. One interesting string manipulation problem is the identification of anagrams.

An anagram is formed when the letters of a word are rearranged to produce another valid word while maintaining the same character frequency. For example, the words `listen` and `silent` are anagrams.

2.2 Motivation

The motivation behind this project is to strengthen understanding of fundamental programming concepts such as:

- String manipulation
- Sorting algorithms
- Pattern recognition
- Data organization
- Algorithm optimization

Grouping anagrams is not only an academic exercise but also a practical problem used in areas like information retrieval, dictionary systems, and search engines.

This project provides hands-on implementation experience while improving analytical and logical problem-solving skills.

3 Design & Algorithms

3.1 Design Approach

The system is designed to classify words based on their character composition. Each word is converted into a canonical representation by sorting its characters alphabetically.

Words producing identical sorted strings are grouped together because they contain the same set of characters.

The implementation uses arrays and string comparison operations to maintain grouped outputs efficiently.

Algorithm (Grouping Anagrams)

Input: n words

1. Read number of words n
2. Store all words in array
3. For each word:
 - a. Sort characters alphabetically
 - b. Store sorted form
4. Compare sorted strings
5. If sorted strings match:
 group corresponding words
6. Print grouped anagrams
7. Stop

3.2 Sorting Logic

The sorting operation is the core component of the algorithm. Every word is converted into a sorted character sequence.

Example:

- listen $\rightarrow$ eilnst
- silent $\rightarrow$ eilnst

Since both sorted forms are identical, the words belong to the same anagram group.

3.3 Time Complexity

For a word of length k , sorting requires:

$$O(n \cdot k \log k)$$

where:

- n = number of words
- k = maximum word length

The algorithm performs efficiently for moderate input sizes.

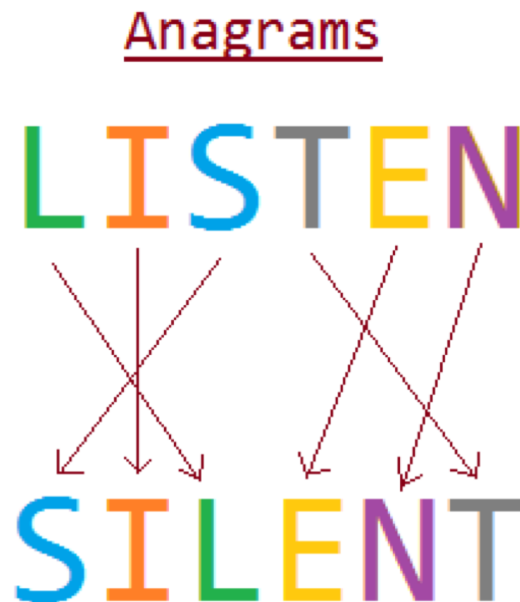


Figure 1: Anagram Example

4 Implementation Details

4.1 Programming Language Used

The project is implemented using the C programming language because of its efficiency and strong support for arrays and string operations.

4.2 Data Structures Used

The following data structures are used:

- One-dimensional arrays
- Two-dimensional character arrays
- Strings
- Boolean visited array

4.3 Working Procedure

The program follows the below sequence:

1. Read the number of words
2. Store all input words
3. Sort characters of each word
4. Compare sorted strings
5. Group matching words
6. Display grouped outputs

4.4 Key Functional Components

- Input handling
- Character sorting
- String comparison
- Grouping mechanism
- Output formatting

4.5 Assumptions

- All words are lowercase
- Maximum word size is limited
- Input size is manageable

4.6 Limitations

- Large datasets may increase execution time
- Special characters are not handled
- Case-insensitive matching is not implemented



Figure 2: Anagram Implementation

5 Test Plan & Results

5.1 Testing Methodology

The program was tested using different input scenarios to verify correctness and reliability.

The following cases were considered:

- Normal input
- No anagrams
- All words as anagrams
- Single-word input
- Mixed length words

5.2 Test Case 1

```
1 Input :  
2 6  
3 listen  
4 silent  
5 rat  
6 tar  
7 god  
8 dog  
9  
10 Output :  
11 listen silent  
12 rat tar  
13 god dog
```

5.3 Test Case 2

```
1 Input :  
2 4  
3 cat  
4 dog  
5 bird  
6 fish
```

```
7 |
8 | Output:
9 | cat
10 | dog
11 | bird
12 | fish
```

5.4 Test Case 3

```
1 | Input:
2 | 3
3 | eat
4 | tea
5 | ate
6 |
7 | Output:
8 | eat tea ate
```

5.5 Result Analysis

The implementation successfully grouped all valid anagrams and produced organized outputs for every test case.

The algorithm correctly handled both standard and edge cases while maintaining acceptable execution efficiency.

6 Analysis & Discussion

6.1 Performance Analysis

The implemented approach performs efficiently for moderate datasets.

The dominant computational operation is sorting. Since each word requires sorting individually, the overall complexity becomes:

$$O(n \cdot k \log k)$$

The solution avoids unnecessary repeated comparisons by using sorted canonical forms.

6.2 Discussion

The project demonstrates how string manipulation and sorting techniques can be combined to solve classification problems effectively.

The implementation is simple, readable, and suitable for beginners while still demonstrating important algorithmic concepts.

Possible improvements include:

- Hashing-based optimization
- Case-insensitive matching
- Unicode character support
- Large-scale dataset handling

6.3 Applications

Grouping anagrams has practical applications in:

- Spell checking systems
- Search engines
- Dictionary applications
- Natural language processing
- Text classification systems

7 Conclusion & Future Work

7.1 Conclusion

This project successfully implements an efficient system for grouping words into anagram collections using sorting and string comparison techniques.

The developed solution demonstrates the practical application of arrays, strings, and sorting algorithms in solving real-world text processing problems.

The implementation produces accurate grouped outputs and provides strong understanding of algorithmic design principles.

7.2 Future Work

Future enhancements may include:

- Further optimization using hashing techniques
- GUI implementation
- Case-insensitive processing
- Large dataset optimization
- Multi-language support

8 Appendix & Code Snippets

8.1 Character Sorting Function

```
1 void sortString(char str[]) {
2     int i, j;
3     char temp;
4     int len = strlen(str);
5     for(i = 0; i < len - 1; i++) {
6         for(j = i + 1; j < len; j++) {
7             if(str[i] > str[j]) {
8                 temp = str[i];
9                 str[i] = str[j];
10                str[j] = temp;
11            }
12        }
13    }
14 }
```

Sorts characters of a string alphabetically using comparison-based swapping.

8.2 Grouping Logic

```
1 for(i = 0; i < n; i++) {
2     if(visited[i] == 1)
3         continue;
4     printf("%s ", words[i]);
5     visited[i] = 1;
6     for(j = i + 1; j < n; j++) {
7         if(strcmp(sorted[i], sorted[j]) == 0) {
8             printf("%s ", words[j]);
9             visited[j] = 1;
10        }
11    }
12    printf("\n");
13 }
```

Compares sorted representations and groups matching words together.

8.3 Sample Input and Output

```
Enter number of words: 5
Enter words:
ate
eat
ant
tan
tea

Anagram Groups:
ate eat tea
ant tan
```

Figure 3: Output 1

```
Enter number of words: 5
Enter words:
listen
silent
earth
heart
alert

Anagram Groups:
listen silent
earth heart
alert
```

Figure 4: Output 2

9 References

- Data Structures Laboratory Manual
- Online educational resources for string processing algorithms
- YouTube lectures on sorting algorithms and anagram grouping
- AI-based tools such as ChatGPT and Gemini used for conceptual understanding, report structuring, and code development support.